



**UNIVERSIDAD PRIVADA DE TACNA**

**FACULTAD DE INGENIERÍA**

**Escuela Profesional de Ingeniería de Sistemas**

**HireCode- *SIMULADOR DE ENTREVISTAS  
TÉCNICAS***

Curso: *PATRONES DE SOFTWARE Sec. A*

Docente: *PATRICK JOSE CUADROS QUIROGA*

Integrantes:

***Diego Fernando Castillo Mamani (2022073895)***

***Sergio Alberto Colque Ponce (2022073503)***

***Renzo Antonio Antayhua Mamani (2022073504)***

**Tacna – Perú  
2026**

| CONTROL DE VERSIONES |           |              |              |            |                  |
|----------------------|-----------|--------------|--------------|------------|------------------|
| Versión              | Hecha por | Revisada por | Aprobada por | Fecha      | Motivo           |
| 1.0                  | MPV       | ELV          | ARV          | 10/10/2020 | Versión Original |

# Sistema HireCode-***SIMULADOR DE ENTREVISTAS TECNICAS***

## Documento de Arquitectura de Software

Versión *{1.0}*

| CONTROL DE VERSIONES |           |              |              |            |                  |
|----------------------|-----------|--------------|--------------|------------|------------------|
| Versión              | Hecha por | Revisada por | Aprobada por | Fecha      | Motivo           |
| 1.0                  | MPV       | ELV          | ARV          | 10/10/2020 | Versión Original |

|                                                                       |          |
|-----------------------------------------------------------------------|----------|
| <b>1. INTRODUCCIÓN</b>                                                | <b>5</b> |
| 1.1. Propósito (Diagrama 4+1)                                         | 5        |
| 1.2. Alcance                                                          | 5        |
| 1.3. Definición, siglas y abreviaturas                                | 5        |
| 1.4. Organización del documento                                       | 5        |
| <b>2. OBJETIVOS Y RESTRICCIONES ARQUITECTONICAS</b>                   | <b>5</b> |
| 2.1.1. Requerimientos Funcionales                                     | 5        |
| 2.1.2. Requerimientos No Funcionales – Atributos de Calidad           | 5        |
| <b>3. REPRESENTACIÓN DE LA ARQUITECTURA DEL SISTEMA</b>               | <b>6</b> |
| 3.1. Vista de Caso de uso                                             | 6        |
| 3.1.1. Diagramas de Casos de uso                                      | 6        |
| 3.2. Vista Lógica                                                     | 6        |
| 3.2.1. Diagrama de Subsistemas (paquetes)                             | 7        |
| 3.2.2. Diagrama de Secuencia (vista de diseño)                        | 7        |
| 3.2.3. Diagrama de Colaboración (vista de diseño)                     | 7        |
| 3.2.4. Diagrama de Objetos                                            | 7        |
| 3.2.5. Diagrama de Clases                                             | 7        |
| 3.2.6. Diagrama de Base de datos (relacional o no relacional)         | 7        |
| 3.3. Vista de Implementación (vista de desarrollo)                    | 7        |
| 3.3.1. Diagrama de arquitectura software (paquetes)                   | 7        |
| 3.3.2. Diagrama de arquitectura del sistema (Diagrama de componentes) | 7        |
| 3.4. Vista de procesos                                                | 7        |
| 3.4.1. Diagrama de Procesos del sistema (diagrama de actividad)       | 8        |
| 3.5. Vista de Despliegue (vista física)                               | 8        |
| 3.5.1. Diagrama de despliegue                                         | 8        |
| <b>4. ATRIBUTOS DE CALIDAD DEL SOFTWARE</b>                           | <b>8</b> |
| Escenario de Funcionalidad                                            | 8        |
| Escenario de Usabilidad                                               | 8        |
| Escenario de confiabilidad                                            | 9        |
| Escenario de rendimiento                                              | 9        |
| Escenario de mantenibilidad                                           | 9        |
| Otros Escenarios                                                      | 9        |

## 1. INTRODUCCIÓN

### 1.1. Propósito (Diagrama 4+1)

El presente documento tiene como propósito describir la arquitectura del sistema **Simulador de Entrevistas Técnicas**, empleando el modelo de vistas **4+1**, el cual permite representar la estructura y comportamiento del sistema desde diferentes perspectivas.

Se presenta una visión global y resumida de la arquitectura, identificando los principales componentes, sus interacciones y las decisiones de diseño adoptadas. Asimismo, se establecen las relaciones entre los requerimientos funcionales y no funcionales del sistema, asegurando que la solución propuesta cumpla con las necesidades del usuario final.

En este contexto, se han priorizado atributos de calidad como la **mantenibilidad**, **usabilidad** y **seguridad**, sin dejar de lado el rendimiento del sistema. Se ha optado por una arquitectura basada en el patrón **Modelo - Vista - Controlador (MVC)**, la cual permite una adecuada separación de responsabilidades, facilitando la escalabilidad y evolución del sistema.

Además, se consideran decisiones técnicas relevantes como la integración con servicios externos (Google OAuth2 para autenticación y Judge0 para ejecución de código), priorizando la eficiencia en la ejecución y la facilidad de integración sobre la portabilidad total del sistema.

### 1.2. Alcance

El presente documento se centra en la definición de la arquitectura del sistema, haciendo énfasis en la **vista lógica**, la cual describe la estructura interna del sistema, sus componentes principales y las relaciones entre ellos.

Se incluyen también aspectos relevantes de las demás vistas del modelo 4+1, tales como la vista de casos de uso, vista de desarrollo, vista de despliegue y vista de implementación, con el objetivo de proporcionar una comprensión integral del sistema.

Sin embargo, algunos aspectos específicos, como la vista de procesos en profundidad, se abordan de manera general debido a su menor impacto en el contexto actual del proyecto.

Este documento no contempla detalles de implementación a nivel de código fuente, centrándose únicamente en el diseño arquitectónico y la estructura del sistema.

### 1.3. Definición, siglas y abreviaturas

A continuación, se presentan las definiciones, siglas y abreviaturas utilizadas en el documento, ordenadas alfabéticamente:

- **API (Application Programming Interface):** Conjunto de funciones y procedimientos que permiten la comunicación entre diferentes sistemas.
- **DB (Base de Datos):** Sistema de almacenamiento estructurado de información.
- **EJS (Embedded JavaScript):** Motor de plantillas utilizado para generar vistas dinámicas en el sistema.
- **HTML (HyperText Markup Language):** Lenguaje estándar para la creación de páginas web.
- **MVC (Modelo - Vista - Controlador):** Patrón arquitectónico que separa la lógica de negocio, la interfaz de usuario y el control del flujo de la aplicación.
- **MySQL:** Sistema de gestión de bases de datos relacional utilizado en el proyecto.
- **Node.js:** Entorno de ejecución para JavaScript en el servidor.
- **OAuth2:** Protocolo de autorización utilizado para autenticación segura mediante terceros.
- **STDIN (Standard Input):** Flujo de entrada estándar utilizado en la ejecución de programas.
- **STDOUT (Standard Output):** Flujo de salida estándar que muestra los resultados de un programa.
- **UI (User Interface):** Interfaz de usuario que permite la interacción con el sistema.
- **UML (Unified Modeling Language):** Lenguaje estándar para modelar sistemas de software.

#### 1.4. Organización del documento

El presente documento se encuentra organizado de la siguiente manera:

- **Sección 1: Introducción**  
Describe el propósito, alcance, definiciones y estructura general del documento.
- **Sección 2: Objetivos y Restricciones Arquitectónicas**  
Presenta los requerimientos funcionales y no funcionales del sistema, así como las restricciones que influyen en el diseño.
- **Sección 3: Representación de la Arquitectura del Sistema**  
Describe el sistema utilizando el modelo de vistas 4+1, incluyendo diagramas UML como casos de uso, clases, secuencia, componentes y despliegue.
- **Sección 4: Atributos de Calidad del Software**  
Define los escenarios de calidad del sistema, tales como funcionalidad, usabilidad, rendimiento, confiabilidad y mantenibilidad.

Este enfoque estructurado permite comprender de manera progresiva el sistema, desde su contexto general hasta su diseño arquitectónico detallado.

## 2. OBJETIVOS Y RESTRICCIONES ARQUITECTONICAS

El presente apartado define las prioridades de los requerimientos del sistema, tanto funcionales como no funcionales, así como las principales restricciones que influyen en el diseño arquitectónico del proyecto **Simulador de Entrevistas Técnicas**.

La priorización permite establecer el orden de implementación y enfocar los esfuerzos de desarrollo en las funcionalidades críticas del sistema.

## 2.1. Priorización de requerimientos

A continuación, se presenta la priorización de los requerimientos del sistema, clasificados en funcionales y no funcionales.

La prioridad se define como:

- **Alta:** Requerimiento crítico para el funcionamiento del sistema
- **Media:** Requerimiento importante pero no esencial
- **Baja:** Requerimiento opcional o de mejora

### 1.1.1. Requerimientos Funcionales

| ID     | Categoría      | Descripción                                                                                                      |
|--------|----------------|------------------------------------------------------------------------------------------------------------------|
| RNF-01 | Seguridad      | La autenticación debe realizarse exclusivamente mediante tokens OAuth2 verificados con google-auth-library       |
| RNF-02 | Seguridad      | Las credenciales sensibles (BD, puertos, claves) deben almacenarse en el archivo .env, nunca en el código fuente |
| RNF-03 | Rendimiento    | La ejecución del código vía Judge0 debe retornar resultado en menos de 10 segundos                               |
| RNF-04 | Disponibilidad | El sistema debe estar disponible al menos el 95% del tiempo en horario de uso académico/laboral                  |
| RNF-05 | Usabilidad     | La interfaz debe seguir el tema Dark/Glassmorphism, intuitiva para usuarios con perfil técnico                   |
| RNF-06 | Mantenibilidad | El código debe seguir arquitectura MVC clara, con separación entre modelos, vistas y controladores               |
| RNF-07 | Compatibilidad | El sistema debe funcionar correctamente en navegadores modernos: Chrome, Firefox y Edge                          |

|               |               |                                                                                                  |
|---------------|---------------|--------------------------------------------------------------------------------------------------|
| <b>RNF-08</b> | Escalabilidad | La base de datos debe usar pool de conexiones MySQL para soportar múltiples usuarios simultáneos |
| <b>RNF-09</b> | Portabilidad  | El sistema debe poder desplegarse en cualquier servidor con Node.js instalado                    |

#### 1.1.2. Requerimientos No Funcionales – Atributos de Calidad

| ID            | Categoría      | Descripción                                                                                                      |
|---------------|----------------|------------------------------------------------------------------------------------------------------------------|
| <b>RNF-01</b> | Seguridad      | La autenticación debe realizarse exclusivamente mediante tokens OAuth2 verificados con google-auth-library       |
| <b>RNF-02</b> | Seguridad      | Las credenciales sensibles (BD, puertos, claves) deben almacenarse en el archivo .env, nunca en el código fuente |
| <b>RNF-03</b> | Rendimiento    | La ejecución del código vía Judge0 debe retornar resultado en menos de 10 segundos                               |
| <b>RNF-04</b> | Disponibilidad | El sistema debe estar disponible al menos el 95% del tiempo en horario de uso académico/laboral                  |
| <b>RNF-05</b> | Usabilidad     | La interfaz debe seguir el tema Dark/Glassmorphism, intuitiva para usuarios con perfil técnico                   |
| <b>RNF-06</b> | Mantenibilidad | El código debe seguir arquitectura MVC clara, con separación entre modelos, vistas y controladores               |
| <b>RNF-07</b> | Compatibilidad | El sistema debe funcionar correctamente en navegadores modernos: Chrome, Firefox y Edge                          |
| <b>RNF-08</b> | Escalabilidad  | La base de datos debe usar pool de conexiones MySQL para soportar múltiples usuarios simultáneos                 |
| <b>RNF-09</b> | Portabilidad   | El sistema debe poder desplegarse en cualquier servidor con Node.js instalado                                    |

#### 2.2. Restricciones

El sistema presenta las siguientes restricciones técnicas y operativas:

- El sistema depende de la disponibilidad de la API externa **Judge0** para la ejecución de código, lo que puede afectar el rendimiento en caso de fallos del servicio.
- La autenticación depende del servicio externo **Google OAuth2**, por lo que el acceso al sistema requiere conexión a internet.
- Se debe utilizar obligatoriamente **Node.js** y **Express.js** como entorno backend, según los lineamientos del proyecto.
- La base de datos está implementada en **MySQL remoto**, lo que puede generar latencias en consultas.
- El uso del motor de plantillas **EJS** limita la interactividad en comparación con frameworks frontend modernos.
- El sistema está sujeto a limitaciones de planes gratuitos de servicios externos (como Judge0 o hosting).
- Se requiere conexión a internet constante para el funcionamiento completo del sistema.
- El sistema está orientado a navegadores modernos, no garantizando compatibilidad con versiones antiguas.

### 3. REPRESENTACIÓN DE LA ARQUITECTURA DEL SISTEMA

#### 3.1. Vista de Caso de uso

La vista de caso de uso del sistema **Simulador de Entrevistas Técnicas** describe las funcionalidades principales del sistema desde la perspectiva del usuario, identificando los actores involucrados y las interacciones que estos realizan con el sistema.

Esta vista permite comprender el comportamiento funcional del sistema y sirve como base para el diseño de la arquitectura, ya que define los escenarios más relevantes que deben ser soportados por la solución.

Los actores principales identificados son:

- **Usuario/Candidato:** Persona que accede al sistema para resolver retos técnicos.
- **Reclutador:** Usuario encargado de gestionar bancos de preguntas y crear retos.
- **Sistema externo (Google OAuth2):** Servicio utilizado para la autenticación.
- **Sistema externo (Judge0):** Servicio encargado de ejecutar el código enviado por el usuario.

Los casos de uso más relevantes del sistema, considerados críticos por su impacto arquitectónico, son:

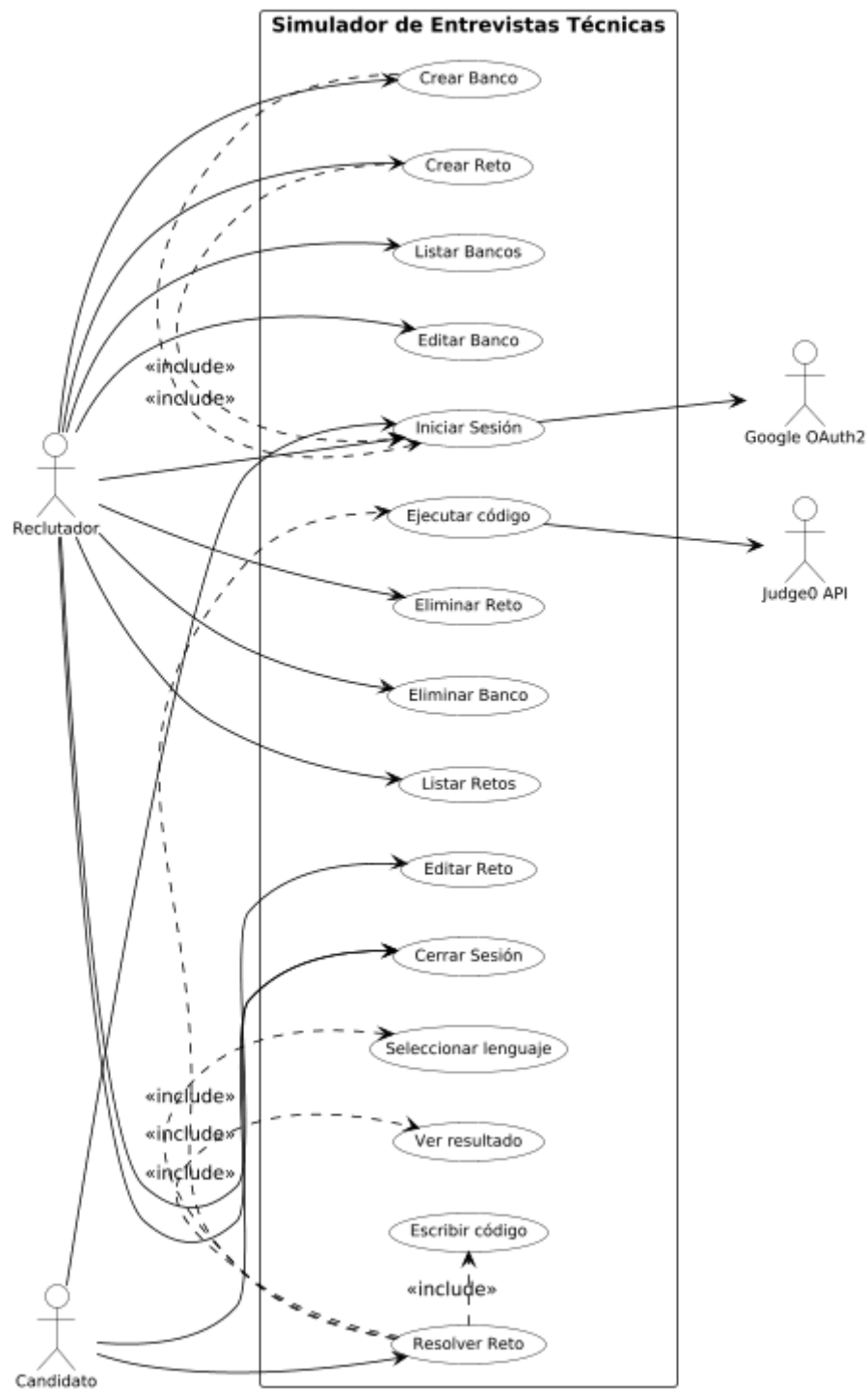


- Iniciar sesión con Google
- Gestionar bancos de preguntas
- Gestionar retos técnicos
- Resolver retos técnicos

Estos casos de uso representan las funcionalidades centrales del sistema y abarcan la mayor parte de la interacción entre los usuarios y la plataforma.

### 1.1.3. Diagramas de Casos de uso

**Diagrama de Casos de Uso - Simulador de Entrevistas Técnicas**



## 3.2. Vista Lógica

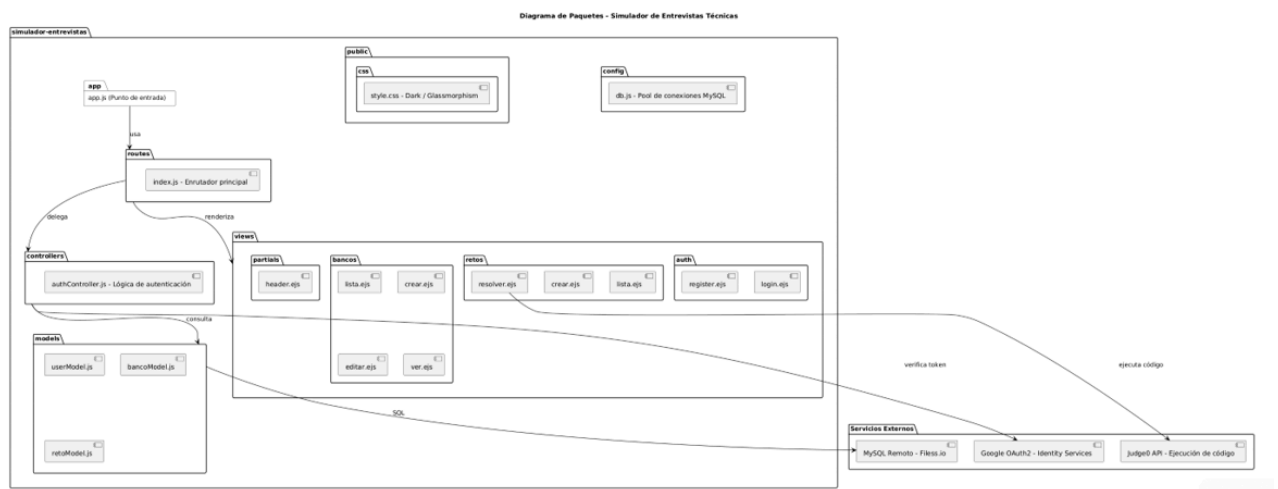
La vista lógica del sistema **Simulador de Entrevistas Técnicas** describe la estructura interna del sistema desde una perspectiva funcional, mostrando cómo se organizan los componentes principales y cómo colaboran entre sí para satisfacer los requerimientos del sistema.

Esta vista se enfoca en representar los elementos clave del diseño, tales como **clases**, **subsistemas**, **paquetes y sus relaciones**, permitiendo comprender la arquitectura del sistema sin entrar en detalles de implementación.

El sistema sigue una arquitectura basada en el patrón **Modelo - Vista - Controlador (MVC)**, lo que permite una adecuada separación de responsabilidades:

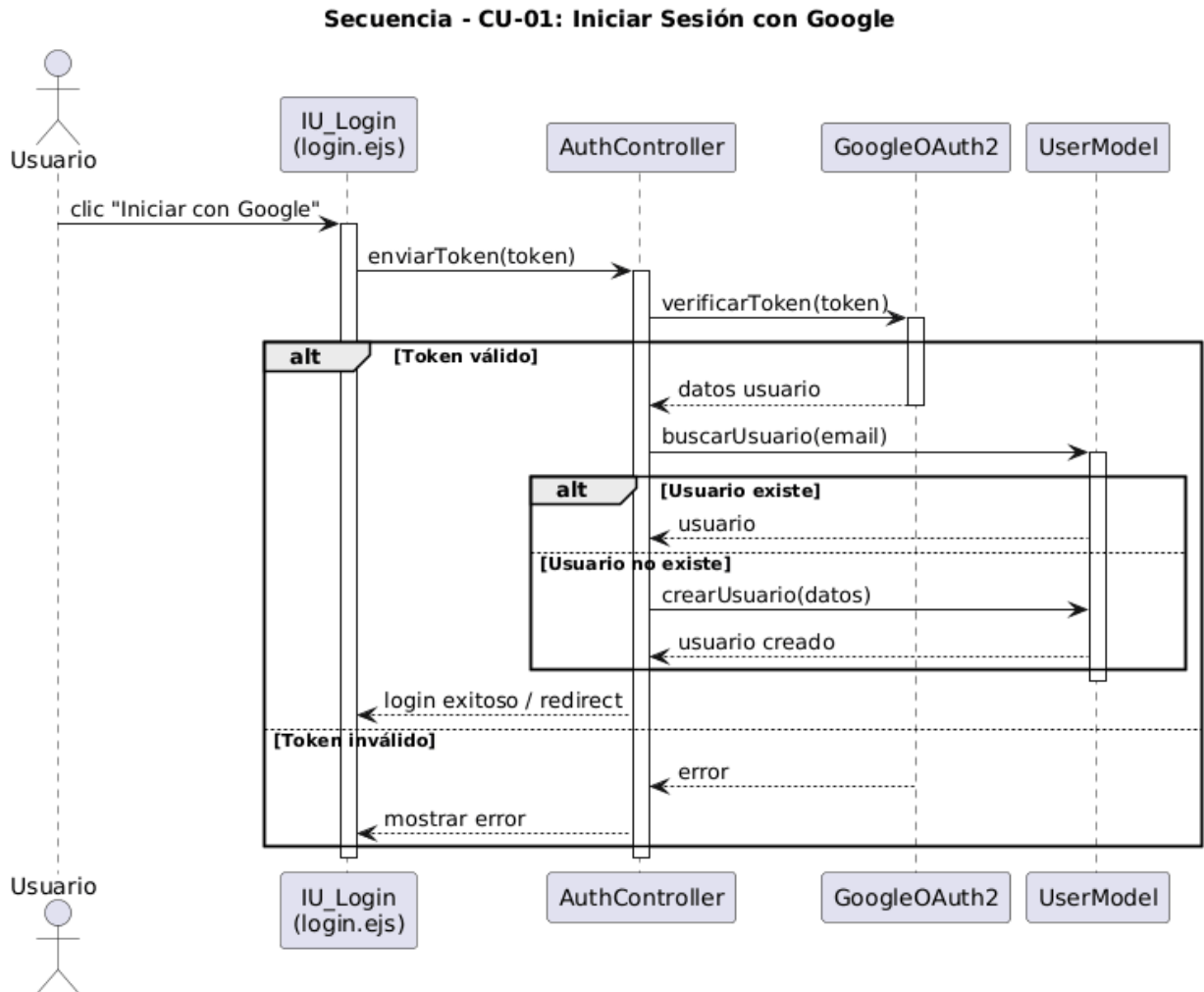
- **Modelo (Model):** Encargado de la gestión de datos y acceso a la base de datos.
- **Vista (View):** Responsable de la presentación de la información al usuario mediante interfaces gráficas.
- **Controlador (Controller):** Gestiona la lógica de negocio y coordina la interacción entre el modelo y la vista.

### 3.2.1. Diagrama de Subsistemas (paquetes)



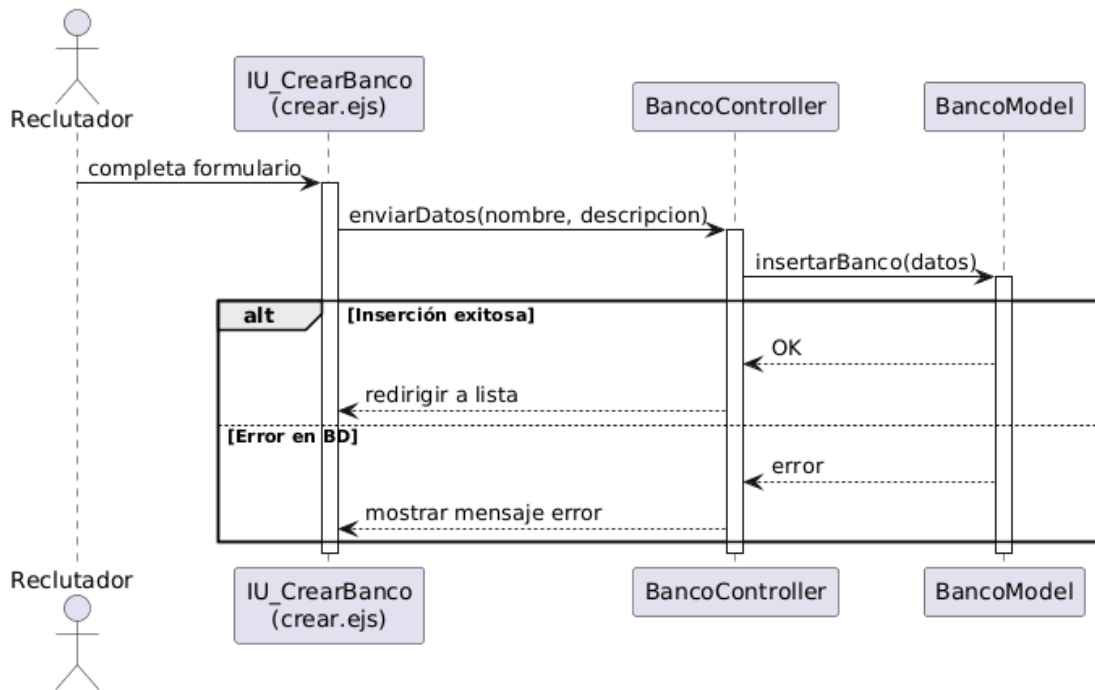
### 3.2.2. Diagrama de Secuencia (vista de diseño)

#### CU-01: Iniciar Sesión con Google



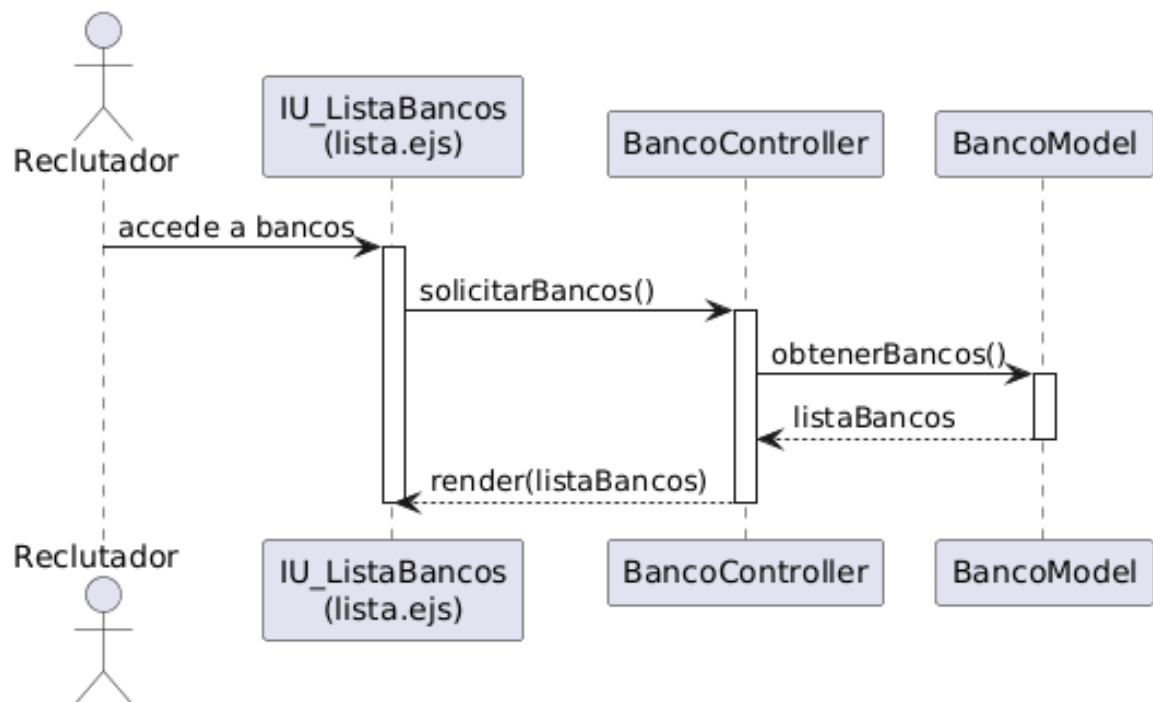
## CU-04: Crear Banco de Preguntas

Secuencia - CU-04: Crear Banco de Preguntas

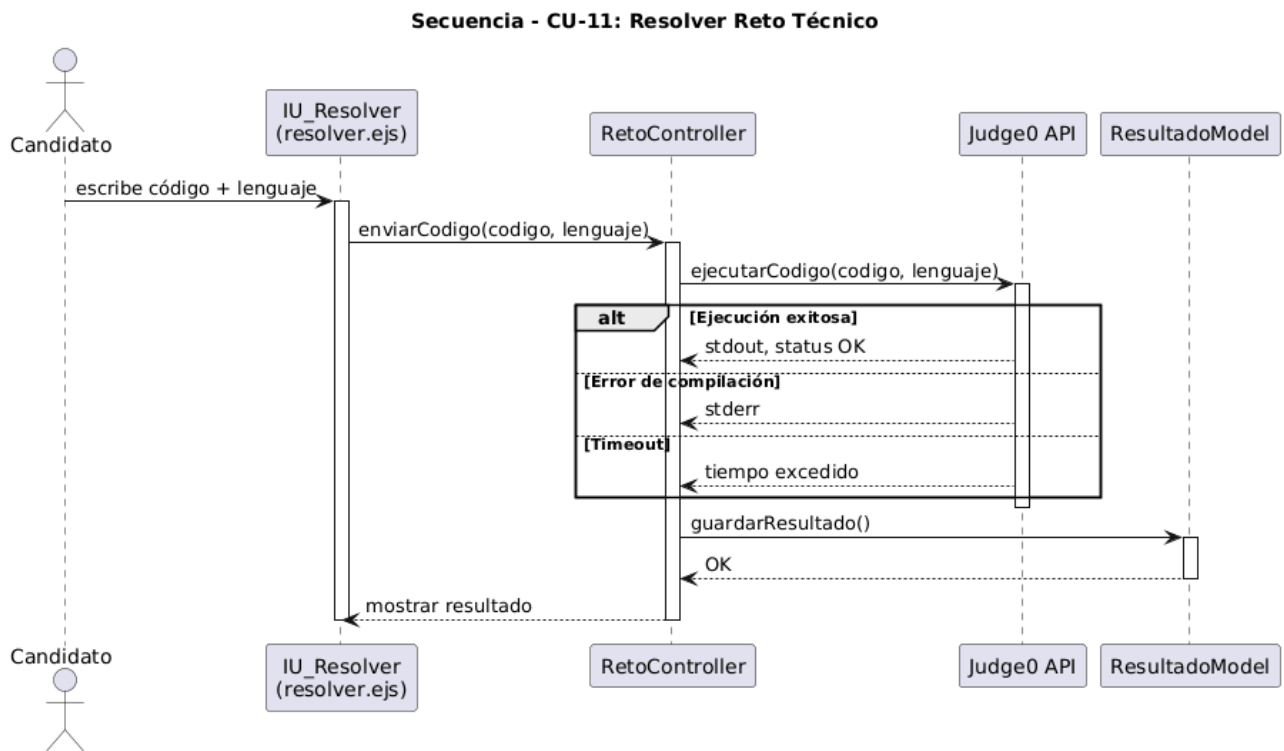


## CU-03: Listar Bancos

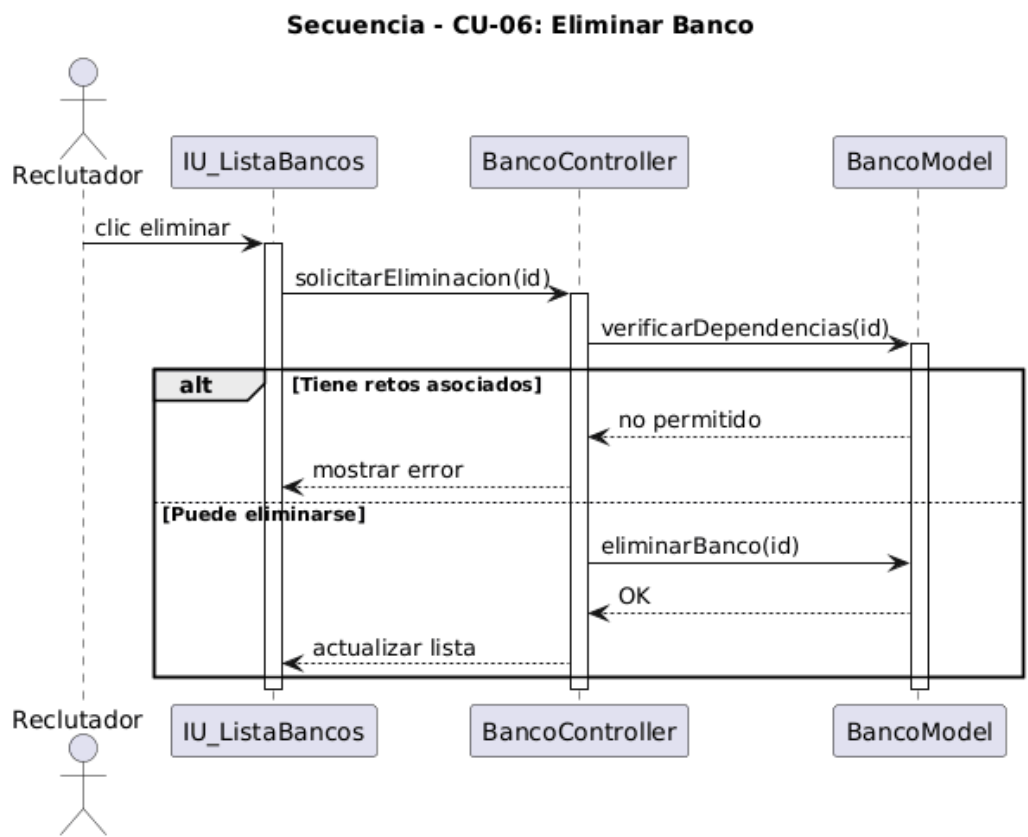
Secuencia - CU-03: Listar Bancos



CU-11: Resolver Reto



CU-06: Eliminar Banco



### 3.2.3. Diagrama de Colaboración (vista de diseño)

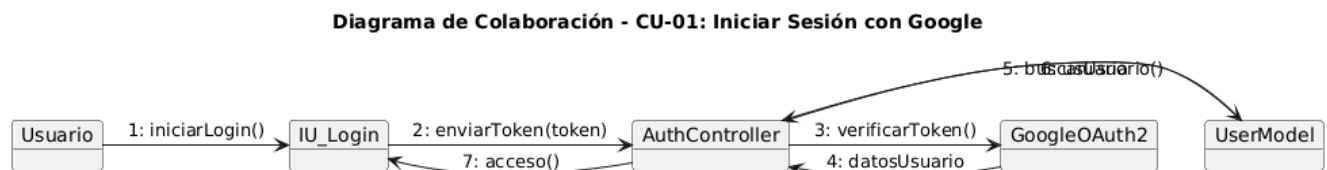
El diagrama de colaboración, también conocido como diagrama de comunicación, representa la interacción entre los objetos del sistema, mostrando cómo se relacionan entre sí para cumplir un caso de uso específico.

A diferencia del diagrama de secuencia, este tipo de diagrama se enfoca en la **estructura de las interacciones**, destacando las conexiones entre objetos y el intercambio de mensajes mediante una numeración que indica el orden de ejecución.

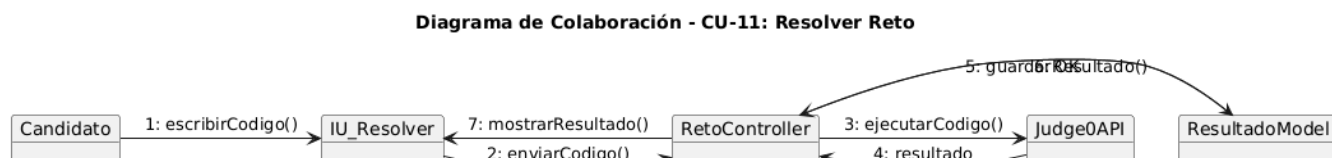
En el sistema **Simulador de Entrevistas Técnicas**, los diagramas de colaboración permiten visualizar cómo los componentes principales (interfaces, controladores, modelos y servicios externos) interactúan para ejecutar funcionalidades clave como la autenticación de usuarios y la resolución de retos técnicos.

Estos diagramas complementan a los diagramas de secuencia, proporcionando una visión más estructural de la comunicación entre objetos del sistema.

#### CU-01 Login



#### CU-11 Resolver Reto



### 3.2.4. Diagrama de Objetos

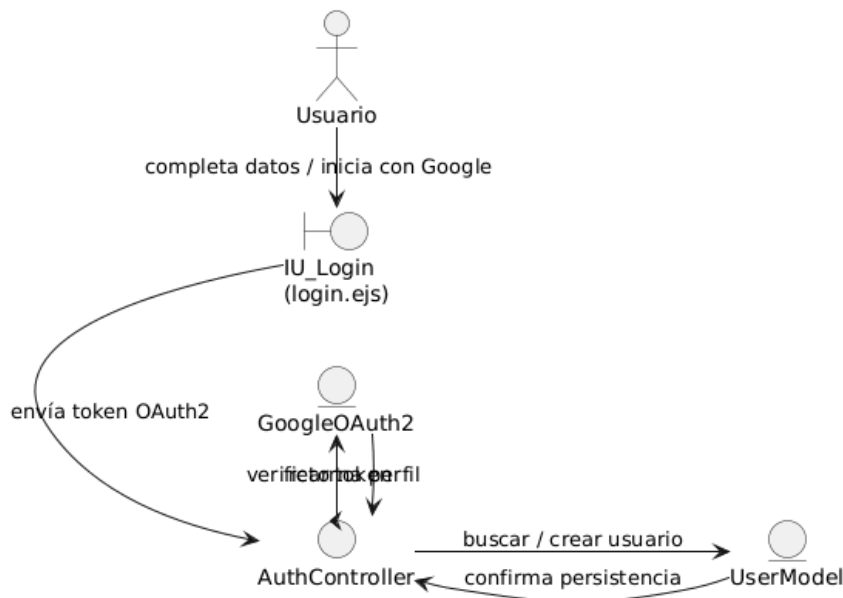
El diagrama de objetos representa una **instantánea del sistema en un momento específico de ejecución**, mostrando instancias concretas de las clases definidas en el diagrama de clases y las relaciones existentes entre ellas.

A diferencia del diagrama de clases, que describe la estructura general del sistema, el diagrama de objetos permite visualizar cómo se materializan dichas clases en situaciones reales, evidenciando valores específicos y vínculos entre objetos en un contexto determinado.

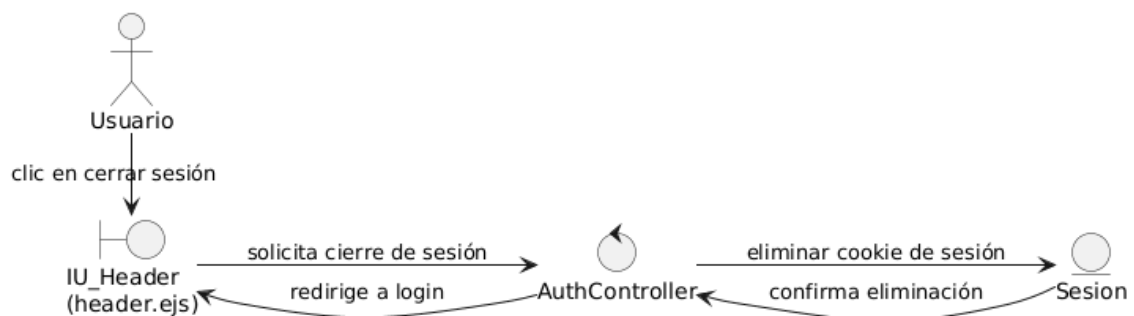
En el sistema **Simulador de Entrevistas Técnicas**, este tipo de diagrama se utiliza para representar escenarios concretos como:

#### a) Análisis de Objetos

##### Análisis de Objetos - CU-01: Iniciar Sesión con Google

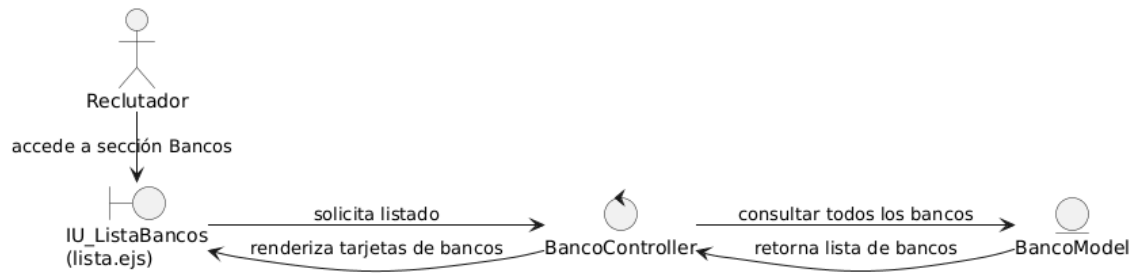


##### Análisis de Objetos - CU-02: Cerrar Sesión

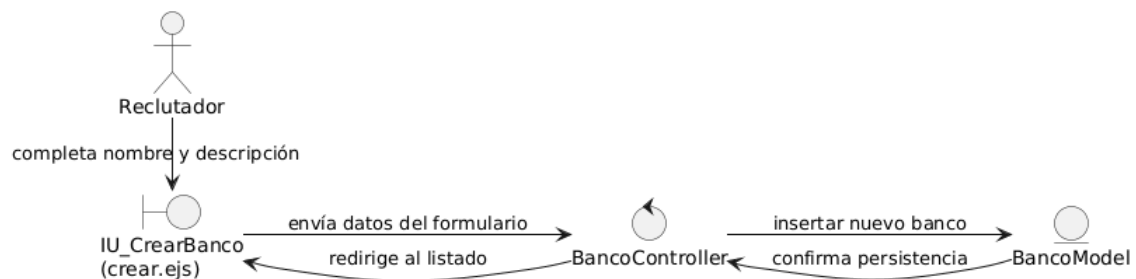




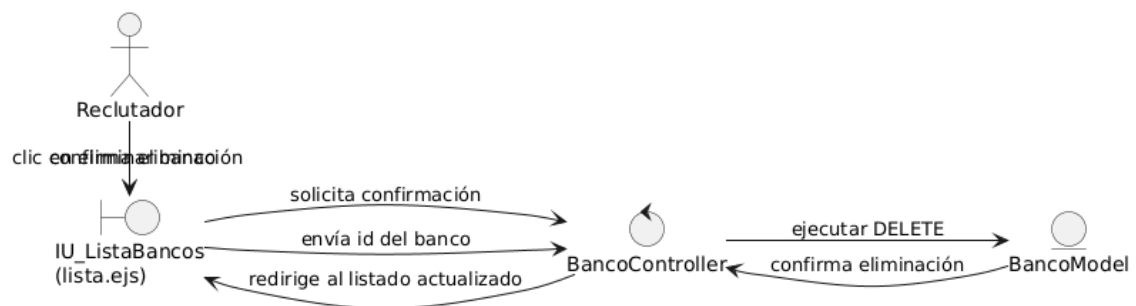
### Análisis de Objetos - CU-03: Listar Bancos de Preguntas



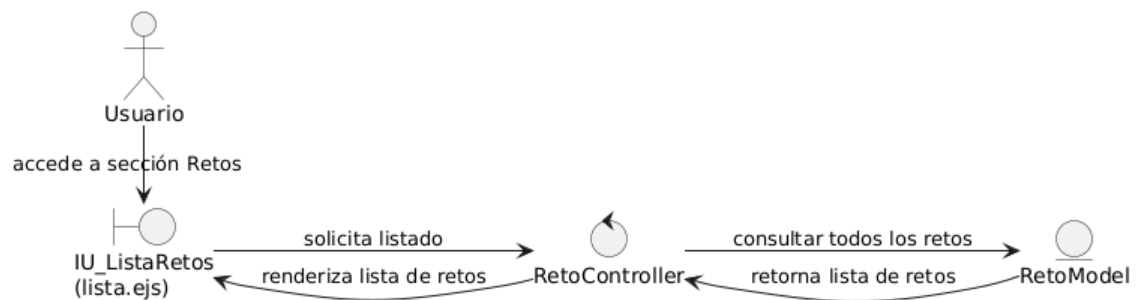
### Análisis de Objetos - CU-04: Crear Banco de Preguntas



### Análisis de Objetos - CU-06: Eliminar Banco de Preguntas

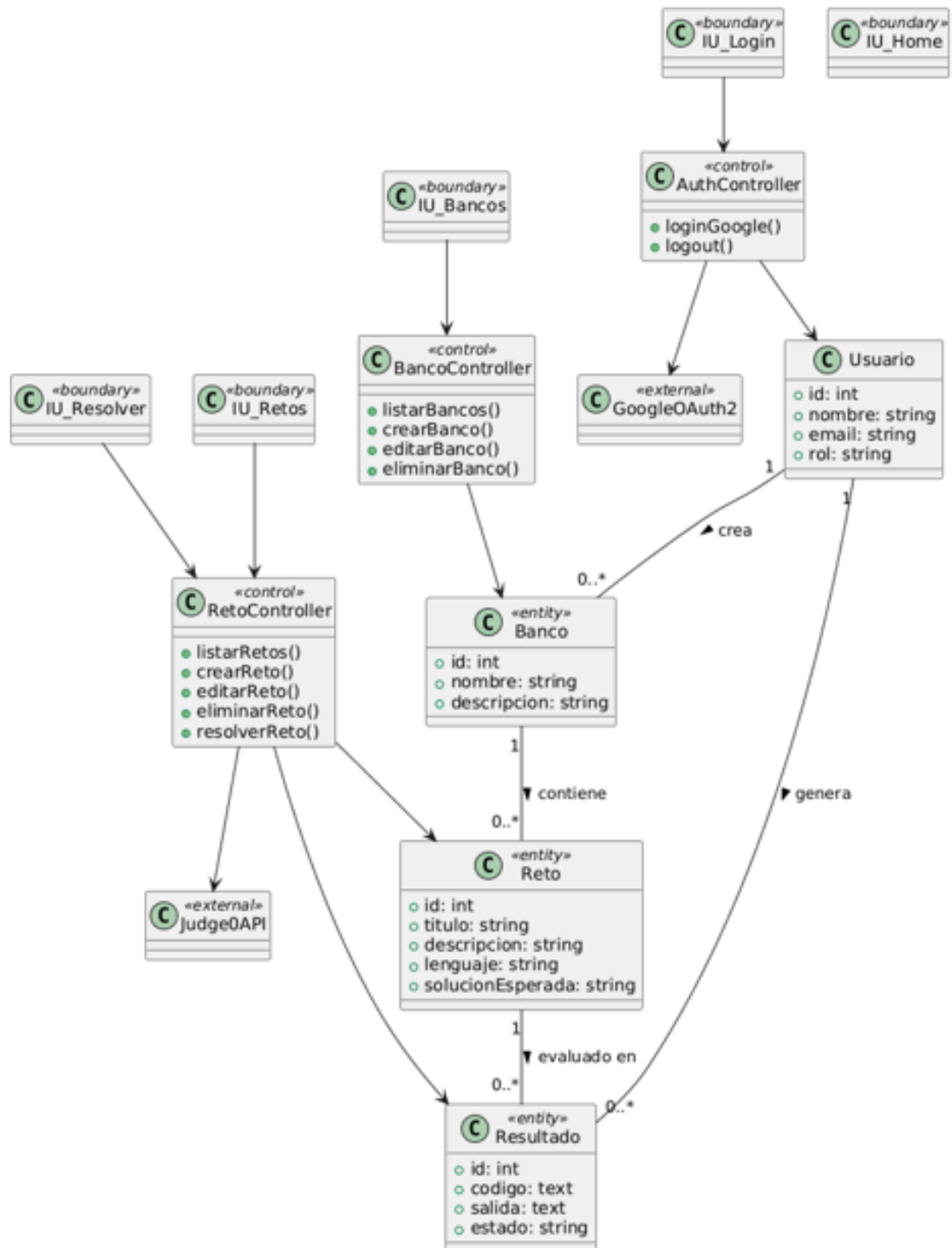


### Análisis de Objetos - CU-07: Listar Retos Técnicos



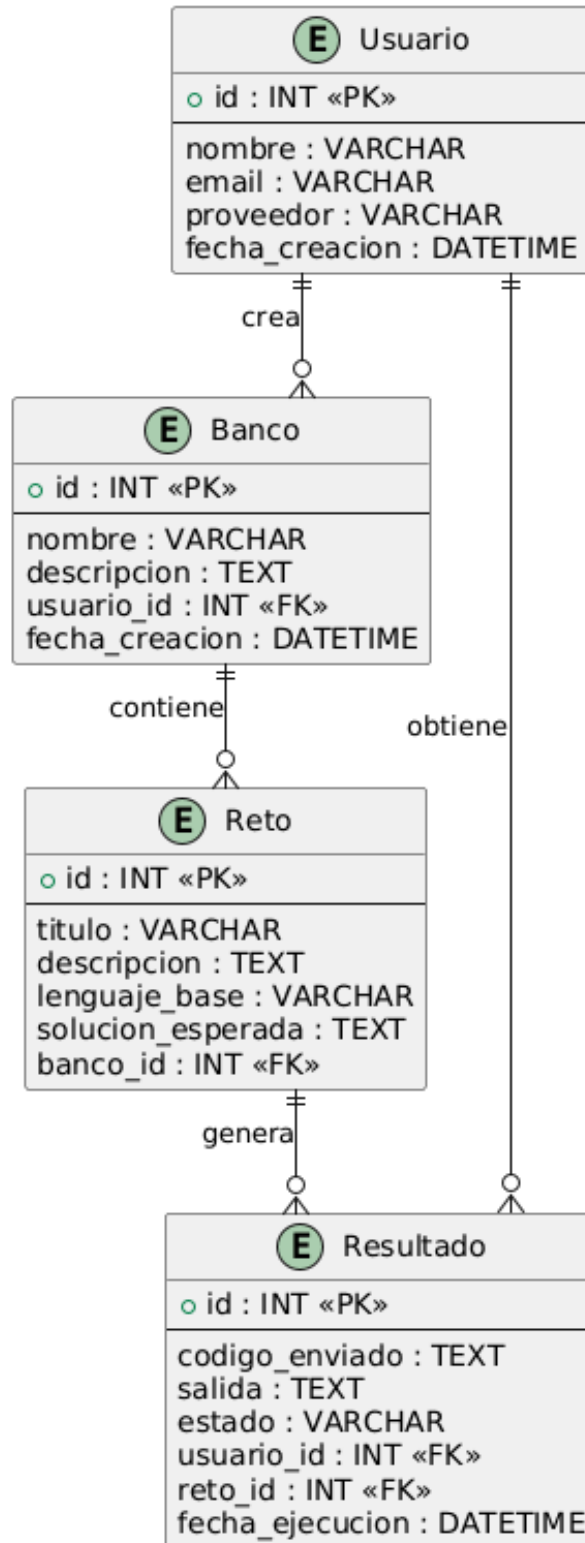
### 3.2.5. Diagrama de Clases

Diagrama de Clases de Análisis - Simulador de Entrevistas Técnicas



### 3.2.6. Diagrama de Base de datos (relacional o no relacional)

#### Diagrama de Base de Datos - Simulador de Entrevistas Técnicas



### 3.3. Vista de Implementación (vista de desarrollo)

La vista de implementación describe cómo los elementos definidos en la vista lógica se materializan en componentes de software reales, organizados en paquetes y módulos dentro del proyecto.

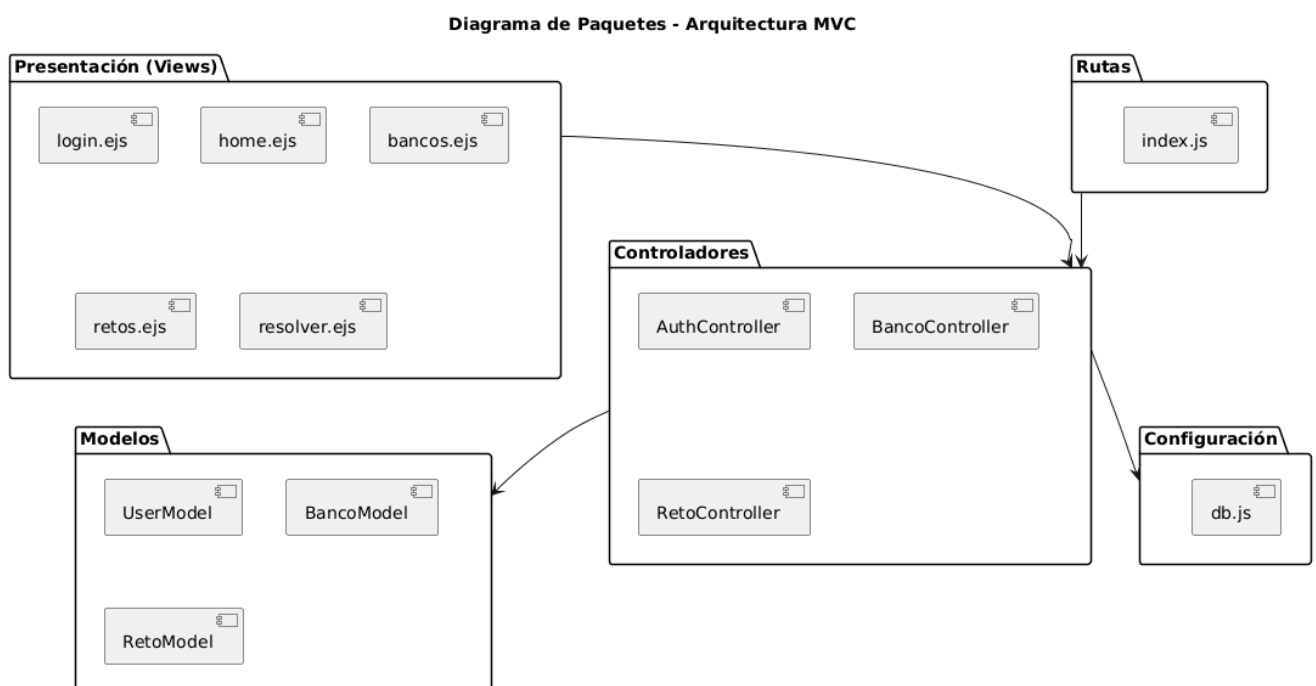
El sistema **Simulador de Entrevistas Técnicas** sigue una arquitectura basada en el patrón **Modelo - Vista - Controlador (MVC)**, donde cada capa está claramente separada y distribuida en directorios específicos del proyecto.

El mapeo entre la vista lógica y la implementación se realiza de la siguiente manera:

- **Subsistema de Autenticación** → **controllers/authController.js** + **models/userModel.js**
- **Gestión de Bancos** → **controllers/bancoController.js** + **models/bancoModel.js**
- **Gestión de Retos** → **controllers/retoController.js** + **models/retoModel.js**
- **Interfaz de Usuario** → **views (EJS)**
- **Configuración** → **config/db.js**
- **Rutas** → **routes/index.js**

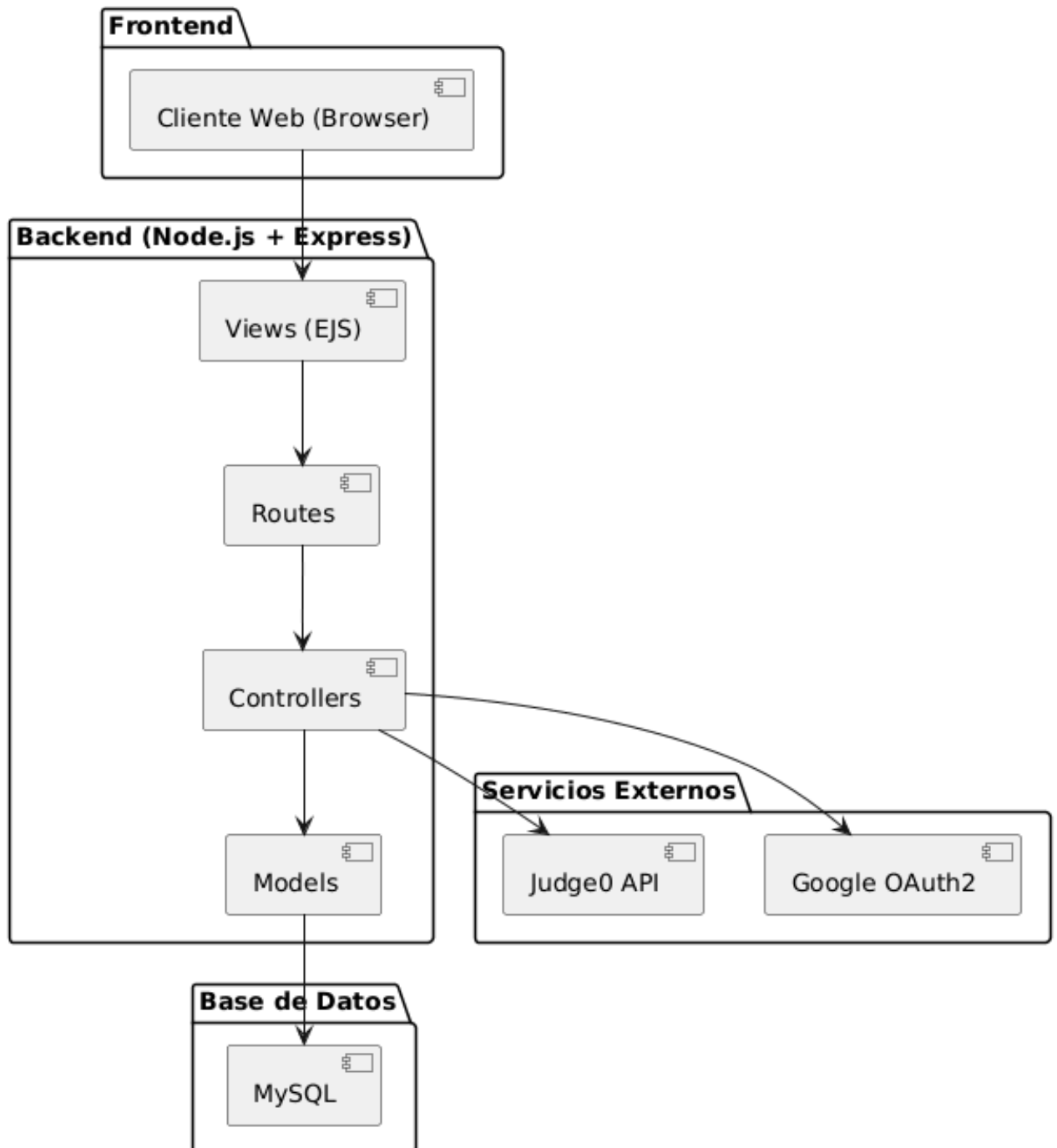
Esta organización permite una estructura modular, facilitando el mantenimiento y escalabilidad del sistema.

#### 3.3.1. Diagrama de arquitectura software (paquetes)



### 3.3.2. Diagrama de arquitectura del sistema (Diagrama de componentes)

#### Diagrama de Componentes - Arquitectura del Sistema



### 3.4. Vista de procesos

La vista de procesos describe el comportamiento dinámico del sistema, identificando los procesos principales y cómo interactúan entre sí.

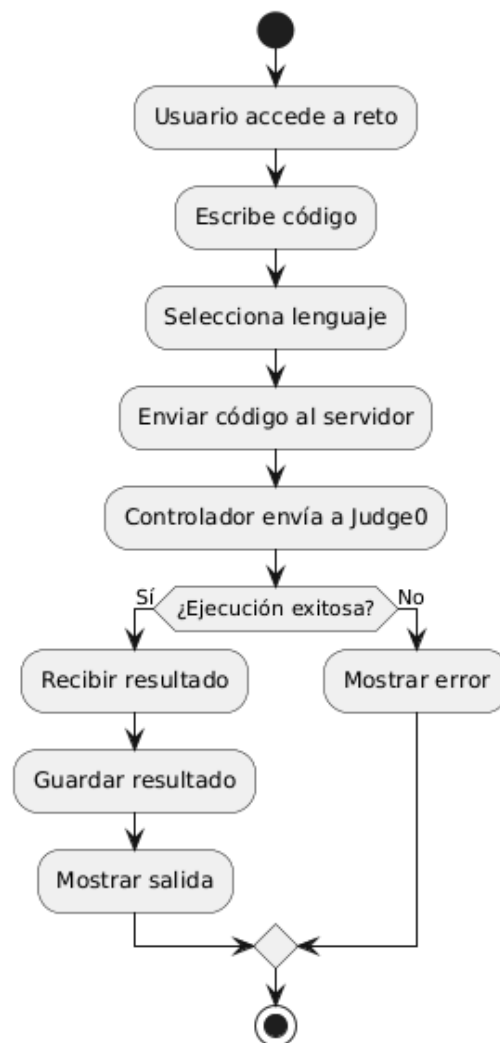
En el sistema, los procesos más relevantes son:

- Proceso de autenticación
- Proceso de gestión de datos (bancos y retos)
- Proceso de ejecución de código

Estos procesos interactúan mediante solicitudes HTTP y llamadas a servicios externos.

#### 3.4.1. Diagrama de Procesos del sistema (diagrama de actividad)

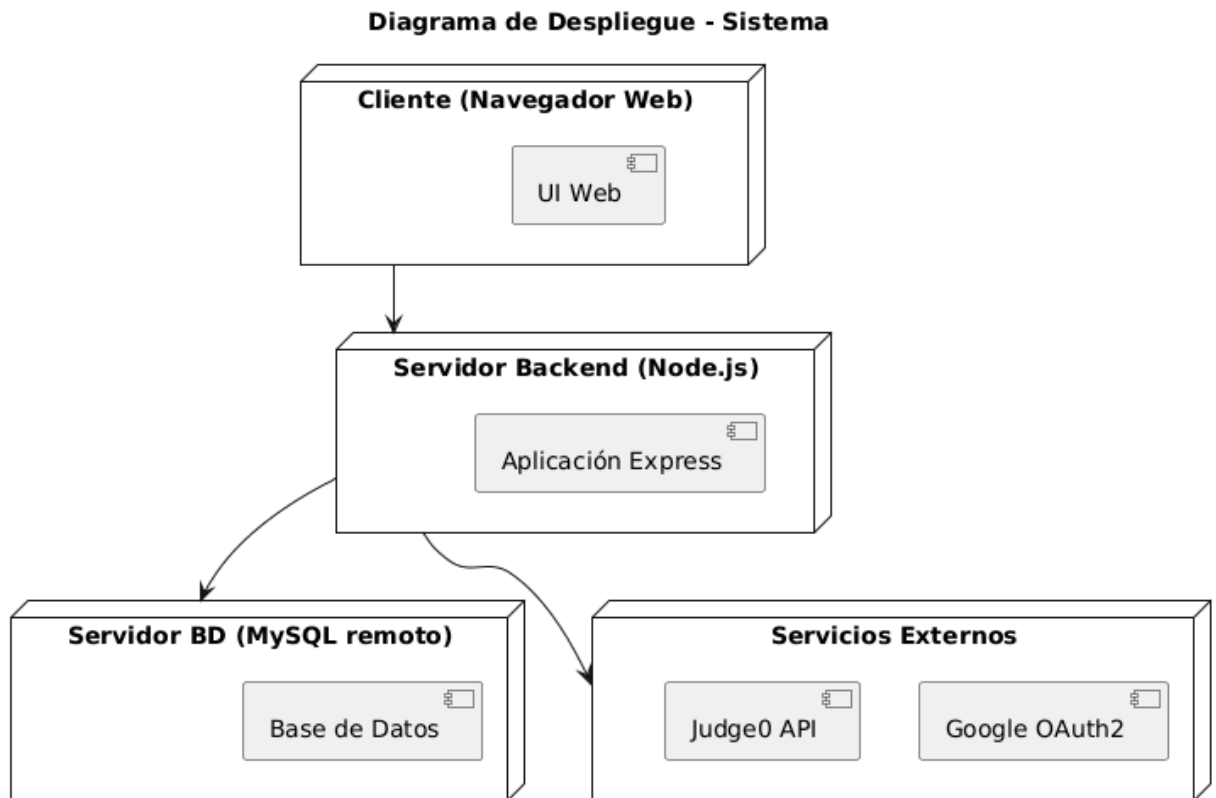
**Diagrama de Actividad - Flujo Resolver Reto**



### 3.5. Vista de Despliegue (vista física)

La vista de despliegue describe la distribución física del sistema, identificando los nodos donde se ejecutan los componentes y cómo se comunican entre sí.

#### 3.5.1. Diagrama de despliegue



## 4. ATRIBUTOS DE CALIDAD DEL SOFTWARE

Los **Atributos de Calidad (QAs)** son propiedades medibles y evaluables de un sistema que permiten determinar el grado en que este satisface las necesidades de los stakeholders. A diferencia de los requerimientos funcionales, los atributos de calidad no describen qué hace el sistema, sino **cómo lo hace**, evaluando aspectos como seguridad, rendimiento, usabilidad y mantenibilidad.

Un sistema puede cumplir correctamente con sus funcionalidades y aun así fallar si no satisface sus atributos de calidad. Por ello, estos atributos son fundamentales en el diseño arquitectónico del sistema **Simulador de Entrevistas Técnicas**, ya que garantizan una experiencia adecuada para los usuarios y una operación eficiente del sistema.

A continuación, se describen los principales escenarios de calidad considerados en el sistema:

### Escenario de Funcionalidad

El sistema debe cumplir con todas las funcionalidades definidas, garantizando que los usuarios puedan:

- Autenticarse correctamente mediante Google OAuth2
- Crear y gestionar bancos de preguntas
- Crear, editar y eliminar retos técnicos
- Ejecutar código en múltiples lenguajes
- Obtener resultados de ejecución de forma clara y precisa

Además, la funcionalidad del sistema se complementa con mecanismos de validación y control de errores, asegurando que las operaciones se realicen de manera correcta y consistente.

### Escenario de Usabilidad

El sistema debe ser fácil de aprender y utilizar, especialmente para usuarios con perfil técnico.

Se consideran los siguientes aspectos:

- Interfaz intuitiva basada en diseño **Dark/Glassmorphism**
- Navegación clara entre módulos (bancos, retos, ejecución)
- Minimización de errores mediante validaciones en formularios
- Presentación clara de resultados de ejecución
- Experiencia de usuario similar a entornos de programación reales



Esto permite que el usuario interactúe de manera eficiente con el sistema, reduciendo la curva de aprendizaje.

### **Escenario de confiabilidad**

El sistema debe garantizar la integridad, disponibilidad y seguridad de la información.

Para ello, se implementan:

- Autenticación segura mediante OAuth2
- Protección de credenciales mediante variables de entorno (.env)
- Manejo de errores en ejecución de código (compilación, timeout, etc.)
- Control de fallos en servicios externos (Google y Judge0)

El sistema busca minimizar fallos y asegurar que los datos se mantengan consistentes incluso ante errores o interrupciones.

### **Escenario de rendimiento**

El sistema debe responder de manera eficiente ante las solicitudes de los usuarios.

Se establecen los siguientes criterios:

- Tiempo de respuesta menor a 10 segundos en ejecución de código
- Uso eficiente de recursos mediante pool de conexiones a la base de datos
- Procesamiento ágil de solicitudes HTTP en el backend

El rendimiento es crítico debido a la naturaleza interactiva del sistema, especialmente en la ejecución de código en tiempo real.

### **Escenario de mantenibilidad**

El sistema debe ser fácil de modificar, extender y mantener.

Se logra mediante:

- Uso del patrón arquitectónico MVC
- Separación clara entre capas (modelo, vista, controlador)
- Organización modular del código
- Uso de buenas prácticas de desarrollo

Esto permite realizar cambios o mejoras sin afectar significativamente otras partes del sistema.

### Otros Escenarios

El sistema debe ser capaz de manejar múltiples usuarios y ejecuciones simultáneas sin degradar significativamente su rendimiento.

Se consideran:

- Escalabilidad mediante uso de pool de conexiones en MySQL
- Capacidad de manejar múltiples solicitudes concurrentes
- Integración eficiente con servicios externos

El atributo de **performance** se refiere a la capacidad del sistema para responder en tiempos adecuados y procesar múltiples eventos en un intervalo determinado, asegurando una experiencia fluida para el usuario.

### CONCLUSIONES

- El Simulador de Entrevistas Técnicas resuelve una necesidad real del mercado tecnológico, ofreciendo una plataforma centralizada y automatizada para la evaluación de candidatos.
- La integración con la API de Judge0 permite la ejecución de código en tiempo real en múltiples lenguajes, garantizando resultados objetivos e inmediatos.
- La arquitectura MVC con Node.js y Express facilita la escalabilidad y mantenimiento del sistema a largo plazo.
- El uso de autenticación OAuth2 con Google garantiza la seguridad del acceso sin necesidad de gestionar contraseñas propias.